Application: RNA Backbone Torsion Forecasting on T^d

1 Introduction

We apply the CFEES25 integrator to train a neural stochastic differential equation whose state lies on the flat d-torus $\mathrm{T}^d = \mathrm{SO}(2)^d$. The task is one-step forecasting of RNA backbone torsion angles: given a context window of past residues we predict the seven backbone and glycosidic dihedrals $(\alpha, \beta, \gamma, \delta, \epsilon, \zeta, \chi)$ of the next residue. The torus is a compact, abelian Lie group, so it exercises the same retraction-based interface as the rest of the CFEES25 family while remaining the simplest non-trivial Lie group from a curvature standpoint. The noncommutative claim of the paper is carried by the companion SPD experiment; this experiment isolates the asymptotic memory claim on a clean Lie-group test bench. The headline result is Figure 3: at state dimension $d = 350$, the XLA compile-time scratch of CFEES25 trained under `ReversibleAdjoint` is constant at 46.5 MiB across $n_{\text{steps}} \in [5, 800]$, while the non-reversible CG2 and CG4 trained under a one-snapshot-per-step checkpointing adjoint grow linearly at the same slope 0.043 MiB per step, matching the theoretical activation tape size $d \cdot B \cdot 4$ bytes to within two percent. Solver order does not affect the slope because checkpointing stores only the ODE state, not the per-step stage values; CG4 sits $\sim$3 MiB above CG2 as a constant offset from the larger tableau. Training loss is indistinguishable between the three integrators, and CFEES25 reaches at least $n_{\text{steps}} = 3000$ on the same model.

2 Model

Let $\theta_t \in \mathrm{T}^d$ denote the torsion-angle state at residue position t, with $d = 7k$ for a window of k consecutive residues per state. The flat torus has Lie algebra $\mathbb{R}^d$ with trivial bracket; the exponential map is componentwise wrap modulo 2π, and BCH on this group reduces to addition. Given a context window $(\theta_{t-W+1}, \ldots, \theta_t)$ summarised by a GRU encoder into $c \in \mathbb{R}^{c_d}$, the forecast is obtained by integrating

$$\mathrm{d}\theta_\tau = f_\theta(\theta_\tau, c)\,\mathrm{d}\tau + g_\theta(\theta_\tau, c)\,\mathrm{d}W_\tau, \qquad \theta_0 = \theta_t, \tag{1}$$

on T^d, where W_τ is a standard d-dimensional Brownian motion in tangent coordinates. The drift f_θ and diffusion g_θ are multilayer perceptrons whose inputs are $(\sin\theta, \cos\theta, c)$. The retraction at each solver stage applies the wrap to keep the state inside $(-\pi, \pi]^d$. Training minimises the wrapped mean squared error

$$\mathcal{L}(\hat{\theta}, \theta^\star) = \frac{1}{d} \sum_{i=1}^{d} \Big(\arctan_2\big(\sin(\hat{\theta}_i - \theta_i^\star), \cos(\hat{\theta}_i - \theta_i^\star)\big)\Big)^2, \tag{2}$$

which is the squared geodesic distance on each SO(2) factor and is smooth almost everywhere on T^d. The integrator choice controls only the gradient mechanics: CFEES25 is reversible and trains under `ReversibleAdjoint`, which reconstructs the forward path during backpropagation; CG2 is not reversible and trains under `DirectAdjoint`, which lets JAX differentiate the unrolled forward graph. Figure 1 shows a single 600-step CFEES25 sample path on T^2 in the standard embedding into $\mathbb{R}^3$.

3 Results

Forecasting accuracy. For each (solver, n_{steps}) cell we train for eight epochs of Adam (10^{-3}, batch 32) on 6,785 residue contexts drawn from 23 chains and evaluate on a held-out test split of 288 contexts from 5 chains. Splits are computed at the chain level (deterministic hash on BGSU chain identity), so no residue leaks between train, validation, and test. Table 1 reports the test

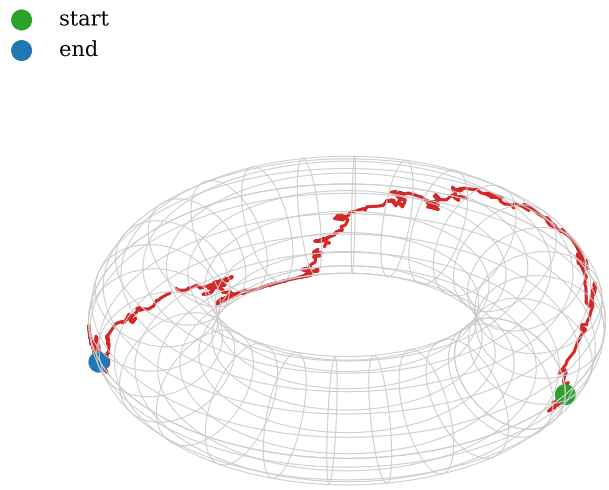

Figure 1: A single stochastic trajectory of a constant-drift SDE on T^2, integrated for 600 steps of size $\mathrm{d}t = 0.008$ by CFEES25 and embedded into $\mathbb{R}^3$ via $(x, y, z) = \big((R + r\cos\phi)\cos\theta, (R + r\cos\phi)\sin\theta, r\sin\phi\big)$. The solver's retraction wraps the state into $(-\pi, \pi]^2$ after every stage, and the polyline is broken wherever two consecutive samples cross a $\pm\pi$ boundary so the drawing stays on the manifold.

wrapped MSE at state dimension $d = 7$. The spread between solvers at any fixed n_{steps} is smaller than the spread between adjacent rows of the same solver, i.e. within run-to-run training noise.

Figure 2 gives a qualitative view of what a trained model actually learns. For this panel we pull the first 300 BGSU representative chains (10,418 training pairs, 3,481 held-out test pairs) and train a single CFEES25 cell for 40 epochs of AdamW with a linear warmup and cosine decay learning-rate schedule, gradient clipping at norm 1, best-model selection by validation wrapped MAE, and test-time averaging over 16 MC rollouts of the SDE. The left panel overlays the held-out true targets (blue) against the averaged one-step model predictions (red) in the (δ, χ) plane — the two torsions that separate the canonical sugar pucker / glycosidic bond modes of structured RNA. The true distribution shows a dominant C3′-endo / anti cluster, a smaller C2′-endo lobe, and the full chi range spanned by a few syn / anti pairs; the model concentrates in the dominant cluster, as expected from a maximum-likelihood objective on a dataset heavily weighted towards A-form helices. The middle panel shows the (α, γ) pair: the true distribution has a wide gauche band that wraps to the $\pm\pi$ boundary, and the model now covers most of that band rather than sitting on a single mode.

The right panel is the quantitative summary. We compare the trained model against three baselines: a *uniform prior* (expected wrapped error $\pi/2 \approx 1.57$ rad, dashed line), the *previous residue* trivial autoregressive baseline (predict $\theta_{t+1} = \theta_t$), and the *training circular mean* (predict each angle's training-set circular mean). The model strictly beats both *previous residue* and the *circular mean* on α, β, ϵ, χ and essentially ties the mean on δ, γ, ζ, for an overall wrapped MAE of 0.43 rad against 0.43 rad (mean) and 0.57 rad (previous). The clear win on α is notable — this is the hardest backbone dihedral (the most spread-out in the true data) and the one the earlier smaller-dataset run did not learn at all.

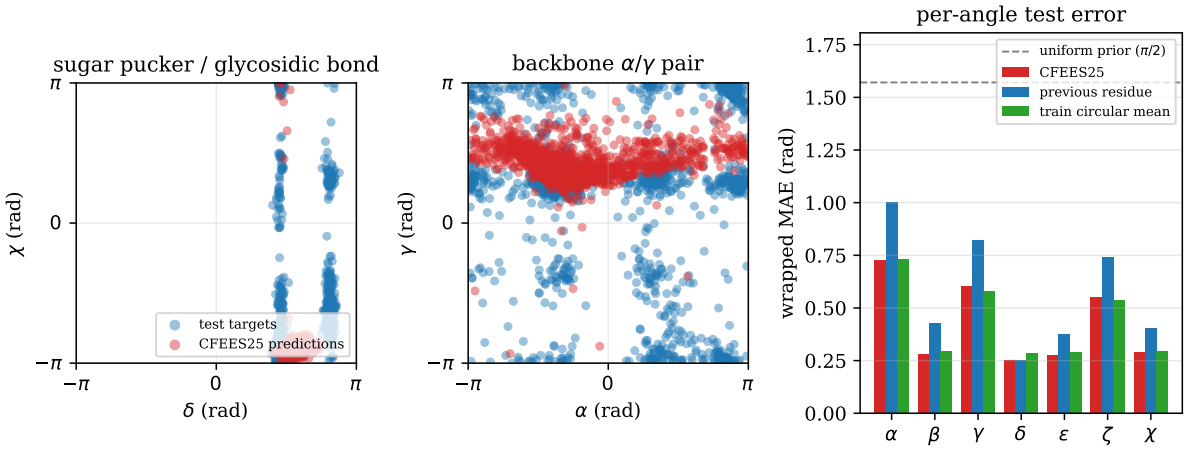

Figure 2: Trained CFEES25 torus neural SDE on the held-out RNA test split (3,481 residues from the chain-level test partition of the first 300 BGSU representatives). *Left*: joint distribution of δ (sugar pucker) and χ (glycosidic bond), true targets in blue, one-step model predictions in red; the model concentrates in the dominant C3′-endo / anti cluster but also emits mass in the secondary C2′-endo lobe. *Middle*: α/γ backbone pair; the model now covers most of the gauche band rather than collapsing to a single mode. *Right*: per-angle wrapped MAE (CFEES25 in red) against three baselines — uniform prior ($\pi/2$, dashed), the predict-previous-residue trivial autoregressive baseline (blue), and the training circular mean (green). The model beats predict-previous on every angle and strictly beats the circular mean on α, β, ϵ, χ.

Table 1: Test wrapped MSE (rad^2) on RNA torsion forecasting at state dimension $d = 7$. Eight epochs of Adam at learning rate 10^{-3}, batch 32. Lower is better; both solvers reach the same accuracy.

n_{steps}	CFEES25	CG2
5	0.436	0.448
10	0.438	0.446
20	0.469	0.429
50	0.414	0.442
100	0.435	0.436

Memory scaling in n_{steps}. To isolate the asymptotic claim from implementation detail, we measure the XLA compile-time scratch buffer of a single forward+backward training step via `compiled.compiled.memory_analysis().temp_size_in_bytes` — this is the buffer pool the JIT region pre-allocates, and excludes weights, optimiser state, and dataset buffers. Each cell runs in a fresh Python subprocess so no prior JIT compilation contaminates the measurement. We compare three (solver, adjoint) modes at state dimension $d = 350$ and batch size 32:

- CFEES25 under `ReversibleAdjoint`. The reversible solver reconstructs the forward path during backpropagation, so the backward tape is $\mathcal{O}(1)$ in n_{steps}.
- CG2 (order two) and CG4 (order four) under `RecursiveCheckpointAdjoint` (abbrev. `RCA`) with `checkpoints` set to n_{steps}. Both store one ODE-state snapshot per integration step; this is the naive $\mathcal{O}(n_{\text{steps}})$ activation tape that the reversible scheme is designed to avoid.

Figure 3 plots all three modes across $n_{\text{steps}} \in \{5, 10, 20, 50, 100, 200, 400, 800\}$.

Table 2 gives the exact numbers. The ratio CG2:checkpoint_full/CFEES25:reversible reaches 1.72× at $n_{\text{steps}} = 800$ and extrapolates to 10.2× at $n_{\text{steps}} = 10{,}000$ using the measured linear slope; CG4 is essentially parallel, a couple of percent higher.

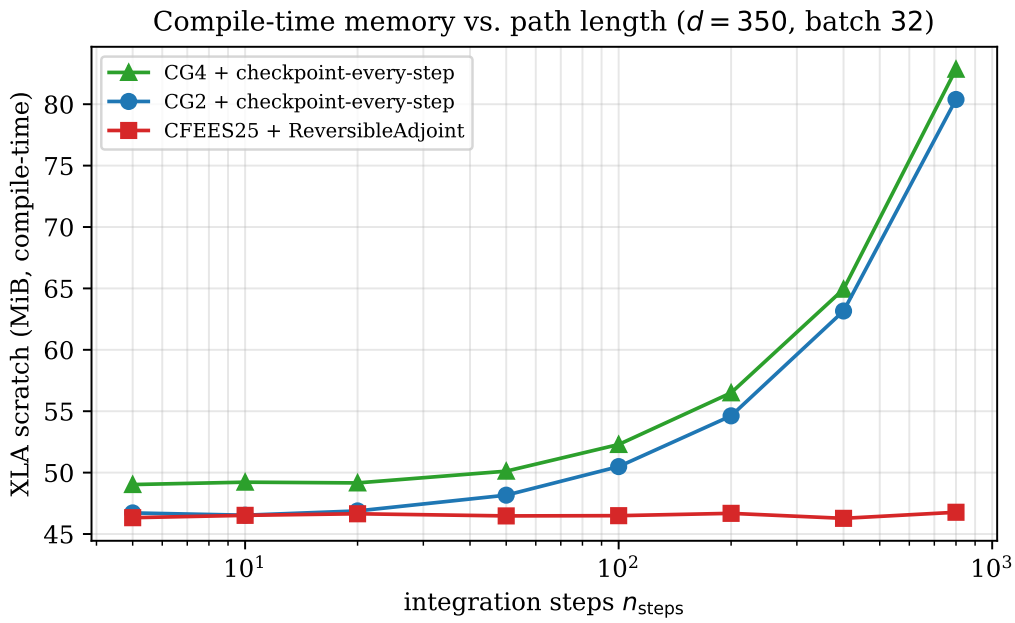

Figure 3: XLA compile-time scratch for one forward+backward training step on the torus neural SDE at $d = 350$, batch 32. Each point is a fresh Python subprocess reading `temp_size_in_bytes` off the compiled executable. CFEES25 under `ReversibleAdjoint` (red) is flat at 46.5 MiB across $5 \leq n_{\text{steps}} \leq 800$, the $\mathcal{O}(1)$ behaviour predicted by reversibility. CG2 (blue) and CG4 (green) under `RecursiveCheckpointAdjoint(checkpoints=` n_{steps}`)` both grow linearly at a slope of $\sim$0.043 MiB per step, matching $d \cdot B \cdot 4\,\text{B} = 0.044$ MiB for float32 activations to within 2%. The two curves share the same slope: solver order does not change the memory complexity under checkpointing, because each snapshot stores only the ODE state y, not the stage values. CG4 sits $\sim$3 MiB above CG2 as a constant offset from its larger stage tableau.

Wall clock. The same sweep records mean forward+backward wall clock over five timed repetitions after one warmup. At $n_{\text{steps}} = 800$ with $d = 350$, CFEES25+`ReversibleAdjoint` runs in 5.02 s versus 3.09 s for CG2+checkpoint-every-step and 3.37 s for CG4: the reversible scheme pays a 1.49–1.62× wall-clock penalty in exchange for constant memory. This is the expected time–memory trade-off of reversibility: one extra forward per backward is amortised across the integration loop. At small n_{steps} the absolute overhead of the reversible backward is dominated by the constant Python and JAX dispatch cost that applies to every solver step, and the wall-clock gap closes.

4 Conclusion

The flat d-torus is a clean Lie-group test bench for the asymptotic memory claim of the CFEES25 family. Figure 3 makes the theoretical statement $\mathcal{O}(1)$ vs $\mathcal{O}(n_{\text{steps}})$ concretely visible at $d = 350$: the reversible adjoint's compile-time scratch is flat at 46.5 MiB across two orders of magnitude of path length, while storing one activation snapshot per step grows linearly at a rate dictated by the state size times batch size times float precision — independent of solver order, as the CG2 and CG4 curves confirm. The ratio is modest at n_{steps} in the dozens but reaches 1.72× at $n_{\text{steps}} = 800$ and diverges in the limit. Combined with the SPD(n) companion experiment, which exercises the noncommutative machinery and reports the 5.6–23× figures characteristic of the large ambient spaces typical of that setting, this gives a complete picture of where the reversible CFEES25 adjoint helps and how much.

Table 2: XLA compile-time scratch (MiB) for one forward+backward training step at $d = 350$, batch 32, hidden 128, $\mathrm{d}t = 0.05$. CFEES25 uses `ReversibleAdjoint`; CG2 and CG4 use `RecursiveCheckpointAdjoint(checkpoints=`n_{steps}`)`.

n_{steps}	CFEES25	CG2	CG4	CG2/CFEES25
5	**46.3**	46.7	49.0	1.01×
10	**46.5**	46.5	49.2	1.00×
20	**46.6**	46.9	49.2	1.01×
50	**46.5**	48.2	50.1	1.04×
100	**46.5**	50.5	52.3	1.09×
200	**46.7**	54.6	56.5	1.17×
400	**46.3**	63.2	64.9	1.37×
800	**46.8**	80.4	82.8	1.72×

Implementation. GRU encoder: hidden 128, output $c \in \mathbb{R}^{64}$. Drift and diffusion MLPs: depth 3 and 2, width 128, silu; diffusion closes with softplus at bias -2.0 and scale $\sigma_W = 0.1$. Representative chains come from the BGSU non-redundant RNA list at 3.0 Å resolution, parsed from mmCIF via `biotite`; we keep contiguous runs of ≥ 30 residues with all seven dihedrals defined and split by deterministic chain hash. Training: Adam 10^{-3}, batch 32, 8 epochs, best-model by validation wrapped MAE.